perINKpheral
Application platform

Datasheet and development guide for applications of the perINKpheral desk display: what the platform gives an application, what it holds it to, and how to write one that belongs on an e-paper panel.

Part I is the datasheet: the numbers and the model. Part II is the guide: how to build an application, with recommendations. Part III is the reference: every call of the host API (ABI 2) and of the SDK library.

Document	revision 1, 2026-11-13
Describes	host API of `sdk/host_api.h` as of 2026-11-13, `abi` 2
Status	preliminary: the host API is not frozen before the first release
Source	github.com/GrakovNe/perinkpheral · MIT

Contents

Part I. Datasheet 4

1. Overview 4
   - Features 4
   - The model in one paragraph 4

2. Characteristics 4
   - Display 4
   - Input 5
   - The sandbox 5
   - Storage 5
   - Network 6
   - Time and background work 6
   - Text 6

3. Execution model 7
   - When an instance exists 7
   - What a handler may count on 7
   - State 7
   - Who owns the screen 7

Part II. Development guide 8

4. A first application 8
   - What is needed 8
   - Create, build, run 8
   - The sample, whole 9
   - Reading the build's report 9

5. The package 10
   - The manifest 10
   - Polls 11
   - Settings 11

6. The screen 12
   - Surface and colour 12
   - Fonts 12
   - Letters beyond ASCII 12
   - Pictures and icons 12
   - The kit 13
   - Asking for the panel 13

7. Buttons 14
   - BACK 14
   - Lists, pickers, dates 14

8. Storage 15
   - The store 15
   - Files 15

9. The network 15

Values picked by the host 15
An answer in pieces 16
A request of the application's own 16
When there is no answer 16
10. Working while away 17
Notifications 17
Alarms, timers 17
11. Time 17
12. Languages 18
13. Design rules 18
It is paper, not a display 18
Memory 19
State and time 19
The user 19
14. Tools and debugging 19
Common problems 20
15. Publishing 20
Part III. Reference 21
16. Return codes 21
17. Exports: what the host calls 21
18. Drawing 22
19. The kit 22
20. The screen, the store, the network 23
21. Time 24
22. The system API 24
23. The SDK library 25
24. Limits 25

Part I. Datasheet

1. Overview

perINKpheral is a desk companion: a 4.2" black-and-white e-paper panel and four buttons on an ESP32-C6. It shows what one glances at - the time, the weather, the next meeting, the state of a home - and stays silent otherwise. The firmware (the *host*) does not change with what the device shows: everything the user sees beyond the menu and the settings is an *application*, installed from a catalog without reflashing.

An application is a WebAssembly module and a manifest. It is written in C against one header, `sdk/host_api.h`, and run by an interpreter (wasm3) inside the host. The host's own pages - Settings, Device, About - are applications written with the same header.

Features

- 400 x 300 pixels, 1 bit, with dithered greys; partial refresh in about half a second
- Four buttons with short and long presses, and their down and up edges
- Sandboxed: one 64 KB page of memory, no pointers into the host, no sockets
- HTTP and HTTPS requests made by the host: scheduled polls and requests of the application's own, to the servers its manifest names
- JSON answers picked apart by the host; any answer streamed in 1 KB pieces
- Runs without being on the screen: answers, alarms and device events load the application for one call
- A key-value store and files of its own that outlive it
- The host's design system: lists, notices, pickers, cells, the row of buttons
- Notifications shown by the host over whatever is on the screen
- Settings declared in the manifest and edited on the device's web page
- Strings in the user's language, loaded by the host; text in Latin with marks, Cyrillic and Greek
- Tools that build, check and measure an application, and drive a device over the network

The model in one paragraph

There is no `main()` and no loop. An application exports handlers with fixed names and the host calls them: a button was pressed, a timer ran out, data arrived, the picture is wanted. A handler changes the application's state and asks for a redraw; the host calls `app_on_draw()` when it needs the frame, and decides itself whether and how the panel is refreshed. One application is in memory at a time. It is unloaded when it leaves the screen, so whatever must be remembered lives in its store, not in its variables.

2. Characteristics

Display

Parameter	Value	Notes
Panel	4.2" e-paper, SSD1683	no backlight; the picture stays without power
Resolution	400 x 300 px	about 120 px per inch
Surface of an application	400 x 260 px	under the host's header; 400 x 300 with `"fullscreen": true`
Depth	1 bit	17 levels of grey by a fixed 4 x 4 dither, for areas only
Partial refresh	0.46 - 0.53 s	only the changed rectangle is sent

Full refresh	2.2 s (2.7 s)	fast waveform (default waveform); the panel blinks
Full refresh after	5 partial ones	a setting of the user's; clears the traces partial refreshes leave
Refresh after a press	at once	
Refresh nobody asked for	at most 1 per 60 s	a setting of the user's (the panel's maker advises 180 s)
A row of pictures	about 1.5 per second	`HOST_REDRAW_NO_BLINK`; traces pile up until the row ends

Input

Parameter	Value	Notes
Buttons	4	in this order on the device: UP, OK, DOWN, BACK
Events of a button	4	key down, key up, pressed, long pressed
Long press	600 ms	reported while the button is still held
Reserved	long BACK	always the host's: empties the stack and shows the menu

The sandbox

Parameter	Value	Notes
Linear memory	64 KB, fixed	one wasm page: 8 KB of stack, static data, the rest for `app_alloc()`; it cannot grow
Module (`app.wasm`)	48 KB at most	executed from flash, not copied into memory
Interpreter's memory	64 KB	about 11 KB for any application plus 4 bytes for every byte of code that has run
Code that can all run	about 13 KB	follows from the line above; `mkapp` warns before the device fails
Time of one handler	3 s	one that runs longer is stopped, and the application with it
Instances in memory	1	of all applications together
Floating point	none	the build refuses a module that has a float operation
C library	none	`sdk/lib` instead: strings, numbers, an arena, `memcpy` and its family
Installed applications	5	per device; a catalog may hold any number

Storage

Parameter	Value	Notes
Store: a key	1 - 15 characters	from `!` to `~`, no spaces
Store: a value	1024 bytes	
Store: all values	4096 bytes	of one application; kept when it is removed, for when it comes back
Files: all of them	120 KB	what came in the package and what the application wrote
Files: a name	1 - 23 characters	`a-z 0-9 . - _`, not starting with a dot

Network

Parameter	Value	Notes
Transport	HTTP, HTTPS	made by the host; an application has no sockets
HTTPS trust	browsers' roots	a server with a certificate of its own making is refused; plain `http` works in a home network
Where an application may ask	4 hosts	named in the manifest, or the server the user gave in a setting
Polls in a manifest	4	each every 10 s to 1 day; on battery not closer than 300 s
Requests at once	1	of the whole device; 2 more may wait
A request's headers / body	1023 / 512 bytes	up to 4 header lines
An answer picked by the host	256 KB	JSON, up to 16 paths, a value up to 63 bytes
An answer in pieces	4 MB	pieces of up to 1024 bytes; about 90 KB/s over plain HTTP
Redirects followed	3	for `GET`; a request of the application's own stays inside its hosts
An HTTPS request	about 6 s	most of it the handshake

Time and background work

Parameter	Value	Notes
Timers (on the screen)	4	one shot, in milliseconds; stop when the application leaves the screen
Alarms (on the screen or off it)	4	in seconds, at a moment, or repeating every 60 s to 1 day; a restart forgets them
Device events (intents)	3 kinds	started, time changed, network up
Notifications waiting	4	of the whole device
Taking the screen	once per 30 s	`request_focus()`; the host decides
Clock	unix seconds	from the network; 0 until first set. Local time and its zone are the host's

Text

Parameter	Value	Notes
Encoding	UTF-8	everywhere; lengths travel with pointers, nothing needs a terminator
Always drawn	printable ASCII, °	
Drawn with a glyph pack	3 packs	Latin with marks, Cyrillic, Greek: added by the user on the device's page
Without the pack	spelled in ASCII	"Moskva", "Strasse", "Athina"; what has no spelling is drawn as `?`
One drawn text	127 bytes	longer is cut between characters
Sizes	6	capitals of 8 to 80 px; two typefaces, the user's choice
Languages of an application	27 known	English built in; others as tables the host loads (section 12)

3. Execution model

An application is not a process. It exists in memory only while the host has a reason to call it, and every such reason is one of its exported handlers.

When an instance exists

What happened	The instance	Handlers, in order
The user opened the application	loaded, stays while it has the screen	`app_init, app_on_focus, app_on_draw ... app_on_blur, app_on_terminate`
An answer, an alarm, an intent or a change of settings, while it is on the screen	the same instance	`app_on_data / app_on_values / app_on_alarm / app_on_intent / app_on_settings`
The same, while it is not on the screen	loaded for the call, unloaded after it; what was on the screen is set aside and returns	`app_init, the handler, app_on_terminate`
It asked for the screen (`request_focus()`) and the host agreed	stays, on top of what was there	`... app_on_focus, app_on_draw`
It left the screen: BACK, the menu, another application on top	unloaded	`app_on_blur, app_on_terminate`

What a handler may count on

- One handler runs at a time, to its end. Nothing of the host or of another application runs meanwhile, so there is nothing to lock.
- It has 3 seconds. A handler still running then is stopped; so is one that traps (a wild pointer, a call the host does not have). The application is ended and the screen says so; the host carries on.
- It returns nothing. What it wants - a redraw, the screen, to leave - it *requests*, and the request takes effect when it returns.
- `app_init()` has run before any other handler of the instance.
- Presses are not lost and arrive in order. Timers of one number collapse into one. The pieces of one answer arrive in order with nothing between them.
- An answer taken off the screen may simply stop: the user pressed a button and the instance was unloaded. `app_on_terminate()` is then the end of it; the poll is asked again in a minute.

State

Globals do not survive an unload, and an unload follows every visit to the screen and every call in the background. Read the state in `app_init()`, write it in `app_on_terminate()` - the last call an instance gets, whatever the reason - or at the moment it changes if it must not be lost to a loss of power.

Do not store a moment as "ticks since the start" and expect it to mean anything later: store `time_now()`, or `time_uptime_ms()` together with `time_boot_id()`, which is another number after every restart.

Who owns the screen

Moving about is a stack. The menu is at the bottom. Opening an application puts it on top; a notification, a question of the host's, an application that asked for the screen go on top of that. BACK is one step down; a long press of BACK empties the stack. Only the top is loaded, so an application that waits under another enters again from what it stored.

Between the surface and the panel stands the host. It compares the frame an application drew with what the panel shows, sends only what differs, and refuses a refresh that comes too soon after the last one that nobody asked for. An application cannot refresh the panel, and cannot leave a half-drawn screen.

Part II. Development guide

4. A first application

What is needed

- **wasi-sdk** 34 or newer, for clang with the wasm32 target; `WASI_SDK_PATH` points at it.
- **Python 3**, standard library only, for the tools.
- A checkout of the **perinkpheral** repository: `sdk/` and `tools/`.
- A **device** on the same network, with a firmware built with the development tools (the default). Its address is on its Device page; export it once as `PERINKPHERAL_HOST`.

Create, build, run

```
tools/devtool/devtool.py new my-app --title "My app"     # a folder with the SDK's sample in it
tools/devtool/devtool.py run my-app                      # build, upload, restart the device into it
tools/devtool/devtool.py serve my-app                    # a page with the live screen, buttons, log, Run
```

`run` takes about ten seconds and does not reflash anything. `serve` is for working beside an editor: the screen as the panel shows it, the four buttons (and the keyboard's arrows, Enter and Backspace), the device's log as it is written, and a button that builds and uploads again.

The sample, whole

```
#include "app_text.h"
#include "host_api.h"

#define MARGIN 16

static const char PRESSES_KEY[] = "presses";
#define PRESSES_KEY_LENGTH ((int32_t)sizeof PRESSES_KEY - 1)

static int32_t presses;

static void draw_line(int32_t line, const char *text)
{
    int32_t line_height = gfx_font_height(HOST_FONT_TEXT);
    gfx_text(MARGIN, MARGIN + line * line_height, HOST_FONT_TEXT, text, app_text_length(text));
}

void app_init(void)                 /* the state comes from the store */
{
    kv_get(PRESSES_KEY, PRESSES_KEY_LENGTH, &presses, sizeof presses);
}

void app_on_terminate(void)         /* and goes back before the instance is unloaded */
{
    kv_set(PRESSES_KEY, PRESSES_KEY_LENGTH, &presses, sizeof presses);
}

void app_on_button(int32_t event)   /* a handler changes the state and asks for a redraw */
{
    switch (event) {
    case BTN_UP_PRESSED:
    case BTN_OK_PRESSED:
    case BTN_DOWN_PRESSED:
        presses++;
        request_redraw(HOST_REDRAW_NORMAL);
        break;
    }
}

void app_on_draw(void)              /* and this paints the state, all of it, on a white surface */
{
    char       counter_buffer[32];
    app_text_t counter;
    app_text_start(&counter, counter_buffer, sizeof counter_buffer);
    app_text_add(&counter, "Presses: ");
    app_text_add_number(&counter, presses);

    draw_line(0, "Hello from wasm");
    draw_line(2, counter_buffer);
}
```

BACK is not handled: a short press of BACK leaves an application that does not take it.

Reading the build's report

```
tools/mkapp/mkapp.py my-app --out build/my-app

build/my-app/app.wasm: 811 bytes, 5 host calls
  module  0.8 KB of 48.0 KB
  code    0.5 KB; running it takes about 13.0 KB of the interpreter's 64.0 KB
  memory  9.1 KB of 64.0 KB: 8.0 KB of stack, 1.1 KB of data; 54.9 KB left for app_alloc()
```

Line	What it measures	What to do when it runs out
`module`	the file: code and initial data. Executed from flash	move pictures and tables into `files/`; draw less by hand, use the kit
`code`	the code alone, and an estimate of what the interpreter needs to run all of it. **This is the scarce one**: a byte of code costs four of memory	see "Memory" in section 13

`memory`	the 64 KB page: the stack, the static data, and what is left for the arena	large buffers into `app_alloc()` under a mark; read files in parts

The build also refuses: an import `host_api.h` does not declare, a float operation (and names it), a `sys_*` call without `"system": true`, a manifest that breaks a limit, a translation that does not match (section 12).

5. The package

```
my-app/
  manifest.json      what the application is, fetches and asks the user for
  my-app.c           the code: any number of .c files
  strings/           optional: en.json, ru.json ... what it says, a language a file      (section 12)
  pictures/          optional: drawings - PNG, PBM, or rows of '#' and '.'               (section 6)
  files/             optional: data installed beside the module, read with file_read()
```

`mkapp` compiles the folder with the SDK's library, turns `strings/` into `strings.h` and `pictures/` into `pictures.h` for the code to include, checks everything against the limits and writes the package: `app.wasm`, `manifest.json`, the files, and a table for every language.

The manifest

```
{
  "name": "plex",
  "title": "Plex Now Playing",
  "version": "0.1.0",
  "abi": 2,
  "hosts": ["{{server}}"],
  "polls": [
    { "url": "{{server}}/status/sessions", "every": "{{every}}",
      "headers": { "Accept": "application/json", "X-Plex-Token": "{{token}}" },
      "pick": ["MediaContainer.Metadata[0].title", "MediaContainer.Metadata[0].grandparentTitle"] }
  ],
  "settings": [
    { "key": "server", "type": "url",    "title": "Server" },
    { "key": "token",  "type": "secret", "title": "Plex token" },
    { "key": "every",  "type": "choice", "title": "Refresh",
      "options": ["15 s", "1 min", "5 min"], "default": "1 min" }
  ],
  "intents": ["started"],
  "icon": [".........."., ".#########.", ".#.......#.", ".#..#....#.", ".#..###..#.", ".#..####.#.",
           ".#..###..#.", ".#..#....#.", ".#.......#.", ".#########.", ".........."]
}
```

Field	Rule	Meaning
`name`	1 - 13 of `a-z 0-9 - _`; the folder's name	the identity: the folder on the device, the key in a catalog, the name of the store. Changing it makes another application
`title`	1 - 23 characters	what the menu and the page call it; translatable
`version`	up to 15 bytes	the developer's own; the page offers Update when a catalog has another
`abi`	`2`	the host API it was built for; another is refused at install
`fullscreen`	true / false (default)	false: the host's header is above, the surface is 400 x 260
`icon`	11 rows of 11 of `#` and `.`	in the menu, on the page, in a catalog. Optional
`polls`	up to 4	what the host fetches on a schedule (below)
`hosts`	up to 4, each up to 63 bytes	where `http_request()` may go: a name, a name with a port, or `{{key}}` of a `url` or `link` setting
`settings`	up to 8	the form on the device's page (below)
`intents`	`started`, `time_changed`, `network_up`	device events the application is loaded for; an unknown one fails the build

`system`	true / false (default)	asks for the `sys_*` calls; the page tells the user before installing
`companion`	an `https` address	a page of the developer's, opened from the settings form, for what a form cannot do

The host skips a key it does not know; `mkapp` checks the ones above.

Polls

Field	Rule	Meaning
`url`	up to 511 bytes; `http` or `https`, or beginning with `{{key}}` of a `url` setting, or exactly `{{key}}` of a `link` setting	`{{key}}` anywhere in it is replaced by the stored setting, encoded for a URL. A poll that needs a setting which is not set fails with `HOST_FAILED_NOT_READY`
`every`	10 to 86400 seconds, or "`{{key}}`" of a setting	with a setting, its value is the pace as a person writes it - "15 min", "1 h", "30 s" - and the user's choice takes effect with no code
`headers`	up to 4; a value up to 255 bytes	`{{key}}` works in values: this is how a token travels
`pick`	up to 16 paths of up to 63 bytes	`current.temperature_2m`, `daily.code[0]`: the host reads the JSON and hands over the values. Without it the answer arrives in pieces

The application knows a poll by its number in the list, from 0.

Settings

A setting is declared, not coded: the device's page draws the form from the manifest without running the application, and stores each value as text under its key in the application's own store. The code reads it with `kv_get()`. A change arrives as `app_on_settings()`, and the host fetches the polls again at once.

Type	On the page	Stored value and notes
`text`	a line of text	as typed, up to 255 bytes
`number`	a number	digits, an optional sign and decimal point
`choice`	one of up to 8 `options`	the option itself, 1 - 23 characters. A translation gives options labels; the stored value does not change
`toggle`	a checkbox	"`1`" or "`0`"; nothing while never touched
`secret`	a password field	the page can write it and never reads it back
`place`	a place found by name	the name; the page fills the two `number` settings named as its `latitude` and `longitude`
`url`	the user's own server	a scheme, a host and maybe a port, nothing after: `http://192.168.1.10:8123`
`link`	a whole address the user was given	`https://host/path?key`, up to 255 bytes; kept from the page like a secret. `webcal://` is taken as `https://`

- `default` is what the page shows while nothing is set. Nothing is stored for it: the code must mean the same when it finds nothing.
- A `url` or `link` is the user's to write and the application's only to read: `kv_set()` on it is refused. When it changes, the secrets that were not given again with it are forgotten - a token is never sent to an address it was not meant for.
- `title` is 1 - 31 characters, `help` up to 95.

6. The screen

Surface and colour

- The origin is the top left of the surface. `gfx_width()` and `gfx_height()` give its size; lay out against them, not against 400 and 260.
- Drawing works only inside `app_on_draw()`, on a surface that arrives white. Anywhere else a `gfx_*` or `ui_*` call returns `HOST_ERR_NOT_DRAWING`. What is drawn outside the surface is clipped.
- `gfx_color()` sets how dark the next calls draw, from `HOST_PAPER` (0) to `HOST_INK` (16). Every `app_on_draw()` starts in ink. Levels between are a dither: for areas and bars. A thin line or small text in grey falls apart.
- The host's margin is `HOST_UI_MARGIN`, 16 px. Keep it.

Fonts

A font is a size. The typeface is the user's choice - Departure Mono, monospaced, or Jersey, whose letters have their own widths - so a layout made by counting characters is wrong for half the users. Measure: `gfx_text_width()`, `gfx_font_height()`, `gfx_font_ascent()`, `gfx_font_cap_height()`.

Size	Capitals, px	Line, px	Characters in 400 px	For
`HOST_FONT_SMALL`	8 / 10	15 / 21	57 / 50	a unit, a caption beside something larger
`HOST_FONT_TEXT`	16 / 15	30 / 28	28 / 33	what is read; the host's own text
`HOST_FONT_MEDIUM`	24 / 25	45 / 43	19 / 21	a title, a value among others
`HOST_FONT_LARGE`	32 / 30	60 / 56	14 / 16	a notice's lines
`HOST_FONT_DISPLAY`	48 / 50	90 / 86	9 / 10	the one value of a screen
`HOST_FONT_HUGE`	80 / 80	150 / 140	5 / 6	a clock: five digits fill the width

The pairs are Departure Mono / Jersey.

- The `y` of `gfx_text()` is the top of the line, not the baseline. To put the top of the capitals at `top`, draw at `top - (ascent - cap_height)`.
- Add `HOST_FONT_TABULAR` to the font for digits that change in place - a clock, a countdown: every digit then takes the room of the widest. Measure with the same flag.
- `gfx_text_fit()` cuts a text to a width with "..." after it, between characters. Never cut text by bytes.

Letters beyond ASCII

Text is UTF-8 and is passed on as it came. The host draws each character by the first of these that applies:

1. its own glyph - printable ASCII always; Latin letters with marks, Cyrillic and Greek when the user has added the pack (Settings, Language, on the device's page);
2. its spelling in ASCII - "Moskva", "Strasse", "Athina", a typographer's dashes and quotes as plain ones;
3. a `?`.

The degree sign is drawn in every size, pack or no pack. Widths differ between the three cases, which is one more reason to measure. Limits on text a person reads - a title, an option - are counted in characters, and a field holds half as many letters of two bytes as of one.

Pictures and icons

The host has twenty icons, 11 cells square: signal, no signal, four arrows, check, cross, plus, minus, trash, gear, clock, hourglass, bell, info, warning, play, pause, update. `gfx_icon()` draws one with a cell of any number of pixels; `ui_icon()` at the host's three sizes, 22, 44 and 99 px.

An application's own drawings go into `pictures/` - a PNG, a PBM, or a text file of rows of `#` and `.`, which may hold several under names:

```
// pictures/things.txt
drop:
.....#.....
....###....
...#####...
..#######..
```

```
#include "pictures.h"   /* made by mkapp */

gfx_picture(x, y, PICTURE_DROP, 4);    /* 44 px: a cell 4 pixels */
gfx_picture(x, y, PICTURE_DROP, 2);    /* 22 px, the same drawing */
```

A picture is a bit a cell, an eighth of what its rows would take as text, and is drawn with a cell of 1 to 16 pixels, so one drawing serves several sizes. Draw on the grid the host's icons use - 11 cells, or 22 and 44 for finer work - and the application's pictures stand beside the host's without looking foreign. A picture too large for the module goes into `files/` and is drawn from there with `gfx_bitmap_file()`, without passing through the application's memory.

The kit

The host's screens are put together from a handful of parts, and an application may ask for the same parts. Nobody has to - `gfx_*` draws anything - but an application built from them looks the part, keeps doing so when the look changes, and is smaller: the kit's code is in the host.

Part	What it is	Use it for
`ui_list_row, ui_list_first, ui_list_visible`	the host's list: a row a line, a value or an icon at the right edge, the chosen row white on a black bar, a scroll bar. With nothing chosen it is a list of facts that UP and DOWN move	anything with more than three items; a menu of the application's own; settings
`ui_cell, ui_cell_fit`	a name, a picture and a value in a box of any size, the value as large as fits	a dashboard: one, two or four values at a glance
`ui_value`	a value and its unit after it on the baseline: "18 km/h"	every number that has a unit
`ui_notice`	two large lines, a line of text, a large icon, what the buttons do	a question, an error, "nothing to show", work going on
`ui_reason`	the notice for what a request came to: "Loading", "Not set up", "No network", "Access denied", "No answer"	every screen that waits for the network
`ui_actions, ui_button`	the row along the bottom: what OK does from the left, what BACK does from the right	every screen where a button does something that is not obvious
`ui_picker`	one of a row of values, the chosen one framed, its neighbours above and below	choosing among a few values with UP and DOWN
`ui_date`	a time, a date or both, set a part at a time	any moment the user has to give
`ui_frame`	a frame with an arrow for each way UP and DOWN can go	marking the one value on a screen that the buttons change
`ui_bar`	how much of how much, filled from the left	a quantity. It changes with the data; it is not an animation
`ui_title, ui_text, ui_icon`	a page's title with its line; text in the host's four styles; an icon at its three sizes	the plain parts of a page

What the buttons do on a list, a picker and a date is in the SDK's library (`app_ui.h`): the application writes only what the result is for (section 7). The catalog's `kit` application shows every part on a page of its own, and its source is the shortest way to see one used.

Asking for the panel

request_redraw(...)	What the host does	For
`HOST_REDRAW_NORMAL`	from a button handler: at once. From a timer, an alarm or data: when the user's policy allows, by default not more than once a minute	nearly everything

`HOST_REDRAW_SOON`	now, although nobody pressed anything	news the user is waiting for: the answer to a refresh they asked for
`HOST_REDRAW_CLEAN`	a full refresh: 2 - 3 s, a blink, no traces	a picture that must be clean; the end of a row of quick ones
`HOST_REDRAW_NO_BLINK`	now, and never as the full refresh the host makes after a few partial ones	stepping through a list; a value that changes every second

There is no budget on redraws: the device is its owner's. Every refresh nevertheless costs battery and a little of the panel's life, and a panel that keeps changing stops being something one can ignore. Section 13 says what follows.

7. Buttons

The buttons mean the same everywhere in the host: UP, OK, DOWN, BACK. Keep the meaning.

A press arrives twice over: as it happens - `KEYDOWN`, then `KEYUP` - and as what it meant - `PRESSED` after the key went up, or `LONG_PRESSED` while it is still held, the release then being only a `KEYUP`. Most applications handle the last two and ignore the rest. One number carries the button and the action, and has a name:

```
void app_on_button(int32_t event)
{
    switch (event) {
    case BTN_OK_PRESSED:       start();  break;
    case BTN_OK_LONG_PRESSED:  reset();  break;
    }
    request_redraw(HOST_REDRAW_NORMAL);
}
```

BACK

The application	A short press of BACK
does not export `app_on_back()`	leaves it: one step down the stack. Nothing to write
exports `app_on_back()`	is delivered there. Go a page back; from the first page call `request_exit()`

`BTN_BACK_PRESSED` never reaches `app_on_button()`, and a long press of BACK is always the host's menu. `request_exit()` is one step down - the user returns to what they were looking at, which matters for an application that took the screen by itself.

Lists, pickers, dates

```
#include "app_ui.h"

static app_list_t list = { .count = 12 };          /* .selected = -1: facts, nothing to choose */

void app_on_button(int32_t event)
{
    switch (app_list_button(&list, event)) {
    case APP_UI_DONE:      open_item(list.selected);               break;   /* OK */
    case APP_UI_CHANGED:   request_redraw(HOST_REDRAW_NO_BLINK);   break;   /* UP, DOWN */
    default:               break;
    }
}

void app_on_draw(void)
{
    int32_t first = app_list_first(&list);
    for (int32_t row = first; row < first + ui_list_visible(&list) && row < list.count; row++) {
        ui_list_row(&list, row, label_of(row), label_length(row), "", 0, HOST_ICON_NONE);
    }
}
```

`app_picker_t` and `app_date_t` work the same way: a struct the application keeps, one call for a button that answers *changed*, *done* or *cancelled*, one call to draw.

8. Storage

The store

`kv_get()` and `kv_set()` keep small values under short keys, in a namespace no other application can reach. It holds the application's state and its settings, and it outlives the application: one that is removed and installed again finds what it left.

```
typedef struct { int32_t version; int32_t minutes; int64_t ends_at; } state_t;
static state_t state;

void app_init(void)
{
    if (kv_get("state", 5, &state, sizeof state) != sizeof state || state.version != 2) {
        state = (state_t){ .version = 2, .minutes = 25 };     /* nothing stored, or an older layout */
    }
}
```

- Give a stored struct a version and check it: the next version of the application reads what this one wrote.
- Every write wears the flash. Write when something changed that is worth keeping, not on every tick.
- `kv_set()` with a length of 0 removes the key.

Files

For what does not fit a value: a list fetched from the network, a cache, a picture. `file_read()` and `file_write()` take an offset, so a file is read and grown in parts without ever being whole in memory. The files of the package's `files/` folder are there from the install.

9. The network

An application never connects to anything. It says what it wants; the host makes the request, one at a time for the whole device, and hands over the answer - whether the application is on the screen then or not. There is no "is the network there": ask, and "no network" is one of the answers.

Way	When	The answer arrives as
a poll with `pick`	on a schedule, with no code running	`app_on_values()`: the values of the paths, once
a poll without	on a schedule	`app_on_data()` in pieces, under the poll's number
`http_request()`	when the code decides: after a press, as a second step	`app_on_data()` in pieces, under the tag the application chose (16 - 255)

Values picked by the host

```
#include "app_values.h"
enum { VALUE_TEMPERATURE, VALUE_CODE };            /* the order of "pick" in the manifest */

void app_on_values(int32_t request, int32_t http_status)
{
    int32_t tenths, code;
    last_status = http_status;                     /* for ui_reason(), when there is nothing to show */
    if (http_status == 200 &&
        app_value_number(VALUE_TEMPERATURE, 1, &tenths) &&      /* "21.46" becomes 215 */
        app_value_number(VALUE_CODE, 0, &code)) {
        keep(tenths, code);                        /* into the store */
    }                                              /* else: what was shown before stays */
    request_redraw(HOST_REDRAW_NORMAL);
}
```

There is no floating point: a number is read with as many decimals as asked for, rounded, as an integer. A path the answer did not have is missing - treat it as no data, not as zero.

An answer in pieces

```
void app_on_data(int32_t request, const uint8_t *data, int32_t length, int32_t flags, int32_t http_status)
{
    if (flags & HOST_DATA_FAILED) {               /* alone, with no data: http_status is a HOST_FAILED_* */
        last_status = http_status;
        return;
    }
    if (flags & HOST_DATA_FIRST) {                /* the HTTP status comes with the first piece */
        last_status = http_status;
        begin_reading();                          /* always start afresh here */
    }
    read_piece(data, length);                     /* valid only until the handler returns */
    if (flags & HOST_DATA_LAST) {
        take_what_was_read();                     /* only now is the answer whole */
        request_redraw(HOST_REDRAW_NORMAL);
    }
}
```

- Read as the pieces come - a state machine, the SDK's `json_pick`, a parser of the application's own - and keep only what will be shown. The answer may be megabytes; the application has 64 KB.
- Replace what is shown only at `HOST_DATA_LAST`. An answer may stop half-way with no last piece and no failure (section 3); `HOST_DATA_FIRST` is always a fresh start.
- `data_response()` gives, inside the handler, what is known of the answer beside its data: the status, Content-Length and -Type, ETag, Last-Modified, Location, Link, Retry-After.

A request of the application's own

```
enum { REQUEST_TOGGLE = HOST_REQUEST_FIRST };
static const char URL[]     = "{{server}}/api/services/light/toggle";
static const char HEADERS[] = "Authorization: Bearer {{token}}\nContent-Type: application/json";

http_request(REQUEST_TOGGLE, HOST_HTTP_POST, URL, sizeof URL - 1, HEADERS, sizeof HEADERS - 1,
             body, body_length);
```

- Only to the manifest's `hosts`. `{{key}}` works in the URL and in header values, so the code never holds the token.
- `HOST_ERR_NO_SPACE`: both waiting places of the device are taken. Ask again later.
- A `GET` is followed through up to three redirects, and never out of the hosts: that fails with `HOST_FAILED_NOT_ALLOWED`. A request with a body is not sent twice: its redirect is the answer.
- `poll_now(n)` fetches a poll now - the user pressed "refresh". `poll_set_interval(n, seconds)` changes its pace from the code, when a setting in the manifest is not enough: a server's Retry-After, say.

When there is no answer

Status	What happened
`HOST_FAILED_NO_NETWORK (-1)`	the device is not on a network
`HOST_FAILED_NO_ANSWER (-2)`	the server did not answer, or not to the end
`HOST_FAILED_NOT_READY (-3)`	a `{{setting}}` the request needs is not set
`HOST_FAILED_NOT_ALLOWED (-4)`	redirected to a host the manifest does not list
`HOST_FAILED_TIMEOUT (-5)`	the server was reached and did not answer in time
`HOST_FAILED_NOT_TRUSTED (-6)`	https, and the certificate is not one the device trusts
`HOST_FAILED_TOO_LARGE (-7)`	over 256 KB to pick apart, or over 4 MB in pieces
`HOST_FAILED_NO_MEMORY (-8)`	the device has no room for the request just now

Keep the last status in the store and hand it to `ui_reason()` when there is nothing to show: it draws the host's notice for every one of these and for the HTTP statuses a user can do something about, and returns 0 for an answer that came, leaving the application to say what is missing then.

10. Working while away

An application that is not on the screen is not running, and yet it can do everything that matters: take data, keep a moment, tell the user. Each of these loads it for one call.

To	Use	Notes
take data on a schedule	a poll in the manifest	needs nothing after a restart: the manifest is the schedule
act at a moment	`alarm_set, alarm_set_at, alarm_set_every`	a restart forgets alarms: set them again on the `started` intent, from a moment in the store
tell the user something	`notify`	the host shows it over whatever is there until it is dismissed, and gives the screen back
show itself	`request_focus`	the application becomes the screen, if the host agrees: once in 30 s at most
keep a line in sight	`header_status`	an icon and a few words in the header above every screen, while the application is away
act when the device starts, joins a network, or its clock is set	an intent in the manifest	`app_on_intent()`

Notifications

```
void app_on_alarm(int32_t id)                       /* loaded for this one call */
{
    host_notice_t notice = { .first = "Time", .second = "is up", .icon = HOST_ICON_BELL };
    notify(&notice);
}
```

The host owns the notification from there: it shows it under its header, keeps it until OK or BACK, and returns the screen to what had it. The application is not loaded for any of that and learns nothing of it, so it keeps no "is showing" of its own. Prefer this to `request_focus()` whenever what there is to say fits two lines.

Alarms, timers

Call	Fires
`timer_set(id, milliseconds)`	once, `app_on_timer(id)`, only while the application is on the screen. Set it again in `app_on_focus()`
`alarm_set(id, seconds)`	once, `app_on_alarm(id)`, on the screen or off it. Counts time, whatever the clock is set to
`alarm_set_at(id, unix_time)`	once, at a moment of the clock; follows the clock when it is set
`alarm_set_every(id, seconds, first_in)`	again and again, every 60 s to a day. Every call loads the application: use the longest period that will do

11. Time

- `time_now()` is unix seconds, and 0 until the device first learns the time. Draw something sensible for that case.
- Everything a person reads is local time, and the zone is the host's: `time_local()` takes the present apart, `time_local_at()` any moment, `time_from_local()` puts one together ("07:30 tomorrow" for `alarm_set_at()`).
- `time_text()` writes a moment in words, in the device's language, so that no application carries the names of days and months: "15:04", "Fri 13 Nov", "Friday", "13 November", and - as near as it is - "15:04", "Tmrw 15:04", "Sat 15:04", "21 Nov".
- To be told when the clock is set, ask for the `time_changed` intent.

12. Languages

What an application says, it says in the language the device is set to. The translations are the developer's - nobody else knows what the words are for - and the host puts them in place.

```
strings/en.json    { "no_events": "No events", "starts_in": "In {0} min" }
strings/ru.json    { "no_events": "Нет событий", "starts_in": "Через {0} мин",
                     "manifest": { "title": "Календарь",
                                   "settings": { "every": { "title": "Обновление",
                                                            "options": { "15 min": "15 мин" } } } } }

#include "strings.h"                              /* made by mkapp: STR_NO_EVENTS, STR_STARTS_IN, app_string() */

const char *text = app_string(STR_NO_EVENTS);     /* the text in the device's language: no call, no copy */

const host_text_argument_t minutes = { .number = 10 };
char line[32];
int32_t length = text_fill(app_string(STR_STARTS_IN), app_text_length(app_string(STR_STARTS_IN)),
                           &minutes, 1, line, sizeof line);          /* "In 10 min" */
```

- `en.json` is required and is built into the module: an application always speaks English. Every other file must have exactly the same keys.
- When an instance is loaded the host copies the table of the device's language over the English one in the application's memory. Reading a string then costs no code.
- `{0}`, `{1}` in a string are filled by `text_fill()` with numbers or texts, in whatever order the language puts them. There are no plural forms: write so that none is needed ("min", not "minutes").
- The key `manifest` in a translation carries what of the manifest a person reads: the title, a setting's title and help, labels for a choice's options. The option itself - what is stored and what the code compares - does not change.
- The build fails on a language the system does not know (`sdk/languages.json`: 27 of them, by two-letter codes), on a translation with a missing or an extra key, and on a letter the language's glyph packs do not have.
- A translation is longer than the English, as a rule by a third. The kit cuts what does not fit with "..."; whether it still reads is the translator's to check on the screen.

13. Design rules

The platform enforces little. These are the rules an application is expected to hold itself to; the host's own screens follow them.

It is paper, not a display

One thing a screen.
A user looks at the device for a second from a metre away. Decide what the one value is, make it the largest thing on the screen, and let everything else be small beside it.

Nothing moves.
No animation, no blinking, no progress bar that advances by itself. A bar shows a quantity and changes when the quantity does. Work going on is said, not shown: a notice with `is_working`.

Change when the data does.
Redraw from a timer only when the picture would differ - a clock once a minute, at the turn of the minute, not once a second. A refresh nobody asked for is limited to one a minute by default; design for that pace, not against it.

Black on white.
Ink on paper, a grey only for areas. White text on a black bar marks the one thing that is chosen or is news.

Say less.
Interface text is short and standard: "Save", "Remove", "Not set up". A label that needs a sentence of explanation is the wrong label.

Memory

Code is the scarce resource, not data.
A byte of code that runs costs four bytes of the interpreter's 64 KB; a byte of data costs one of a page that is mostly empty. Prefer a table to a chain of `if`, the kit to drawing by hand, the host's `pick` to a parser, `time_text()` to a calendar of one's own.

Keep small helpers out of line
when they are called from many places - `__attribute__((noinline))` - and let the build's report tell whether it helped.

Pictures as bits.
In `pictures/`, never as strings in the source; large ones in `files/`.

Never hold an answer whole.
Read it as it comes and keep what will be shown. For a working buffer use `app_alloc()` and give it back with a mark.

State and time

Every instance may be the last.
Anything that matters is in the store before `app_on_terminate()` returns. Test by opening the application, leaving it and opening it again: it must look the same.

Keep what is shown until something better has arrived whole.
A failed or interrupted request is not a reason to show an empty screen in place of an hour-old forecast.

Say why there is nothing.
An empty screen is a bug. "Not set up", "No network", "Loading" are one call: `ui_reason()`.

The clock may not be known, and may jump.
Handle `time_now() == 0`. Use `alarm_set()` for "in ten minutes" and `alarm_set_at()` for "at seven".

The user

Nobody types on four buttons.
Whatever needs text - a token, an address, a city - is a setting in the manifest, edited on the device's page. On the device a value is one of a few, stepped through with UP and DOWN.

Take the screen rarely.
A notification for what the user asked to be told - a timer, a meeting. Never for what can wait until they look.

The user's data goes to the user's servers.
List only the hosts the application needs. A server of the user's own is a `url` setting, not a host in the manifest; a token is a `secret`, sent by the host in a header the code never sees.

Ask the network no more than the data changes.
Give the pace to the user as a setting with a sensible default. On battery the host will not fetch more often than every five minutes whatever is asked.

14. Tools and debugging

Command	What it does
`tools/mkapp/mkapp.py <dir> --out <dir>`	builds one application: compiles, links the SDK's library, checks, measures
`tools/devtool/devtool.py new <dir> [--title T]`	a folder to start from: the SDK's sample under that name
`tools/devtool/devtool.py run <dir> [--shot f.png]`	builds, uploads, restarts the device into the application
`tools/devtool/devtool.py serve [dir]`	a local page: the live screen, the buttons, the log, Run
`tools/devtool/devtool.py shot [f.png]`	the frame the panel shows, as a picture

`tools/devtool/devtool.py press 3 3 2L [--shot f.png]`	presses: 1 - 4 short, 1L - 4L long, in the device's order
`tools/devtool/devtool.py open <name>`	puts an installed application on the screen
`tools/devtool/devtool.py log [--follow] [--app]`	the device's log; `--app` keeps the lines of applications
`tools/devtool/devtool.py set <app> <key> <value>`	a value into an application's store: a setting, without the page
`tools/devtool/devtool.py status`	firmware, network, free memory, why the device last started
`tools/mkpicture/mkpicture.py <files> --show`	shows drawings as rows of # and . : what `mkapp` will make of `pictures/`
`tools/mkvideo/mkvideo.py <video> --out <bin>`	a video into a 1-bit stream small enough to be a file of an application
`tools/catalog/mkcatalog.py <folder> --out <dir>`	builds a folder of applications into a catalog

`host_log(level, text, length)` writes a line of up to 200 characters into the device's log; it is the `printf` there is. `press ... --shot` waits for the panel and saves the frame, which is enough to script a regression test of an application's screens.

Common problems

Symptom	Most likely
Nothing on the screen	drawing outside `app_on_draw()`, or no `request_redraw()` after the state changed
The state is gone after leaving and returning	it lived in globals. Store it in `app_on_terminate()`, read it in `app_init()`
"<title> crashed" with "memory alloca..."	more code ran than the interpreter has room for. The build warned: see "Memory" in section 13
"<title> cannot start", "Update the app."	built against another `host_api.h` than the device's firmware. Rebuild and install again
A poll answers -3	a `{{setting}}` of its URL or headers is not set
https answers -6	a certificate of the server's own making, or expired. Use `http` inside a home network
`http_request()` returns -4	both waiting places of the device are taken; ask again from the next handler
Grey shadows of earlier pictures	a row of `NO_BLINK` redraws that never ended with a `CLEAN` one
Letters as `?` or in Latin spelling	the glyph pack for the alphabet is not installed on the device; or the text was cut by bytes
A press does nothing right after the device starts	an answer is being taken in the background; the press ended it and the screen is being drawn again

15. Publishing

A device installs from *catalogs*: folders of static files with an index, on any hosting that serves HTTPS - a git host's raw files, a static site.

```
tools/catalog/mkcatalog.py my-apps --out catalog       # builds every application of the folder
                                                       # catalog/catalog.json and a folder for each
```

- The index lists each application's name, title, version, icon, and its files with their sizes and SHA-256. The device takes a file only when both match.
- There are no signatures. What vouches for an application is HTTPS to a source the user chose; what contains it is the sandbox.
- The official catalog is the `perinkpheral-apps` repository: it is the first source of every device. A user adds up to three more on the device's page - the address of a repository is enough.
- The page shows a user what an installed application is, from its manifest: where it connects, how often, what it keeps, whether it starts with the device.
- Bump `version` with every change: it is how a device knows there is an update.

Part III. Reference

Every declaration of `sdk/host_api.h`, ABI 2. Numbers are `int32_t` unless shown otherwise. A text argument is a pointer and a length in bytes, UTF-8, with no terminator needed. A call with no value of its own returns `HOST_OK` or a negative `HOST_ERR_*`.

16. Return codes

Code	Meaning
`HOST_OK (0)`	done
`HOST_ERR_NOT_DRAWING (-1)`	a drawing call outside `app_on_draw()`
`HOST_ERR_ARGUMENT (-2)`	a pointer outside the application's memory; an unknown font, icon, timer, style; a size beyond reason
`HOST_ERR_NOT_FOUND (-3)`	nothing stored under the key; no such file; no time yet; a path the answer did not have
`HOST_ERR_NO_SPACE (-4)`	the value is larger than the buffer; the store or the files are full; all waiting requests or notifications are taken
`HOST_ERR_FAILED (-5)`	the storage itself failed

17. Exports: what the host calls

Every one is optional. The host skips what a module does not export.

Export	Called
`void app_init(void)`	once, right after the instance is loaded and its strings are in place
`void app_on_focus(void)`	the application got the screen. Set `timer_set()` timers here
`void app_on_blur(void)`	it is about to lose the screen
`void app_on_terminate(void)`	the last call of an instance, whatever the reason: it left the screen, a call in the background is done, the device restarts, the application is removed. `app_on_blur()` comes first if it was on the screen
`void app_on_draw(void)`	the frame is wanted: paint all of it from the state. The surface is white
`void app_on_button(int32_t event)`	a button: `BTN_<UP|OK|DOWN|BACK>_<KEYDOWN|KEYUP|PRESSED|LONG_PRESSED>`. Never `BTN_BACK_PRESSED` or `BTN_BACK_LONG_PRESSED`
`void app_on_back(void)`	a short press of BACK, for an application with pages of its own
`void app_on_timer(int32_t id)`	a timer ran out
`void app_on_alarm(int32_t id)`	an alarm came, on the screen or off it
`void app_on_data(int32_t request, const uint8_t *data, int32_t length, int32_t flags, int32_t http_status)`	a piece of an answer: to poll number `request`, or to the request under that tag. `flags`: `HOST_DATA_FIRST`, `_LAST`, `_FAILED`
`void app_on_values(int32_t request, int32_t http_status)`	the answer to a poll with `pick`; read the values inside the handler
`void app_on_settings(void)`	the user changed the application's settings on the page
`void app_on_intent(int32_t intent)`	a device event the manifest asked for: `HOST_INTENT_TIME_CHANGED` (1), `_NETWORK_UP` (2), `_STARTED` (3)
`uint8_t *app_data_buffer(void)`	where the host puts a piece of data: 1 KB. Provided by `sdk/lib`

`uint8_t *app_strings_buffer(void)`	where the host puts the strings of the device's language. Made by `mkapp`

18. Drawing

Call	What it does
`gfx_width(void), gfx_height(void)`	the surface: 400 x 260 under the header, 400 x 300 full-screen
`gfx_color(color)`	how dark the calls after it draw: `HOST_PAPER` (0) to `HOST_INK` (16)
`gfx_text(x, y, font, text, length)`	text with the top of its line at `y`; returns the width drawn. At most 127 bytes
`gfx_text_width(font, text, length)`	the width it would take
`gfx_text_fit(font, text, length, max_width, fitted, capacity)`	the text cut with "..." to fit the width; returns its length
`gfx_font_height(font), gfx_font_ascent(font), gfx_font_cap_height(font)`	the whole line; from its top to the baseline; capitals and digits above the baseline
`gfx_rect(x, y, w, h, fill)`	`HOST_RECT_OUTLINE` or `HOST_RECT_FILLED`
`gfx_line(x0, y0, x1, y1)`	a line one pixel wide, both ends included
`gfx_picture(x, y, picture, cell)`	a picture as `mkapp` makes of `pictures/`: a byte of width, a byte of height, a bit a cell; a cell 1 - 16 px
`gfx_icon(icon, x, y, cell)`	one of the host's `HOST_ICON_*`, 11 cells square, a cell 1 - 16 px
`gfx_bitmap(x, y, w, h, bits)`	a bitmap pixel for pixel: a set bit takes the colour, a clear one leaves what is there
`gfx_bitmap_file(x, y, w, h, name, name_length, offset)`	the same, read by the host from a file of the application's at an offset
`gfx_qr(x, y, cell, text, length)`	a QR code of up to 200 bytes; returns its modules a side, and with a cell of 0 only that
`void request_redraw(flags)`	the picture is out of date: `HOST_REDRAW_NORMAL`, `_SOON`, `_CLEAN`, `_NO_BLINK`
`header_text(side, text, length)`	a line of the application's own in the host's header above its screen: `HOST_HEADER_LEFT` or `_RIGHT`, up to 63 bytes. A length of 0 gives the side back

19. The kit

Call	What it does
`ui_text(x, y, style, text, length, max_width)`	text in a style: `HOST_UI_TEXT`, `_TITLE`, `_CAPTION`, `_VALUE`; cut at `max_width` (0: not cut). Returns the width
`ui_text_width(style, text, length), ui_text_height(style)`	to centre and to stack
`ui_title(text, length)`	a page's title with its line; returns the `y` where the content begins
`ui_icon(x, y, icon, size)`	a host's icon at `HOST_UI_ICON_SMALL` (22 px), `_MEDIUM` (44), `_LARGE` (99)
`ui_actions(ok, ok_length, back, back_length)`	the row along the bottom, 44 px (`HOST_UI_ACTIONS_HEIGHT`). A length of 0: that button does nothing here
`ui_button(x, y, name, name_length, action, action_length, mirrored)`	one button's name on its plate with what it does beside it; returns the width

`ui_list_visible(list), ui_list_first(list)`	of a `host_list_t` - count, selected (or -1), top, height (0: to the bottom), first: the rows in sight, and the first of them
`ui_list_row(list, index, label, label_length, value, value_length, icon)`	one row; a row out of sight draws nothing
`ui_notice(notice)`	a `host_notice_t` on the whole surface: first[64], second[64], detail[96], ok[32], back[32], icon, is_working. Empty texts are not drawn
`ui_reason(status)`	the notice for a request's status. Returns 1 when it drew, 0 for an answer that came (2xx)
`ui_value(x, y, font, value, value_length, unit, unit_length)`	a value and its unit on the baseline; returns the width of both
`ui_value_width(font, value, value_length, unit, unit_length)`	the same, measured
`ui_cell(cell, picture)`	a `host_cell_t` - x, y, width, height, name[48], value[48], size - with a picture or 0. A cell that is the whole surface has its picture 88 px square, any other 44
`ui_cell_fit(cell, picture)`	the size the cell would have, 1 the largest. Give every cell of a screen the largest number among them
`ui_bar(x, y, width, value, maximum)`	how much of how much, 14 px high
`ui_frame(x, y, width, height, arrows)`	a frame, with `HOST_UI_ARROW_UP` and `_DOWN` for the ways there are to go
`ui_picker(picker)`	a `host_picker_t`: above[64], chosen[64], below[64], ok[32], back[32]. Draws the row of buttons too
`ui_date(date)`	a `host_date_t`: year, month, day, hour, minute; has (`HOST_DATE_HAS_TIME | _DATE`); selected (`HOST_DATE_HOUR` .. `_YEAR`, or `_NONE`); ok[32], back[32]

20. The screen, the store, the network

Call	What it does
`void request_exit(void)`	done: one step down the stack. From a button, back or timer handler
`void request_focus(void)`	from a handler off the screen: worth showing now. The host decides; once per 30 s at most
`notify(notice)`	a `host_notice_t` for the host to show until dismissed: first (required), second, detail, icon. `HOST_ERR_NO_SPACE` when four wait
`header_status(side, icon, text, length)`	an icon and up to 31 bytes in the header above every screen while the application is away. Both empty gives the side back; a restart clears it
`timer_set(id, milliseconds)`	one shot, id 0 - 3; 0 cancels; setting it again restarts it
`alarm_set(id, seconds)`	id 0 - 3; 0 takes it back
`alarm_set_at(id, int64_t unix_time)`	at a moment of the clock
`alarm_set_every(id, seconds, first_in)`	every 60 - 86400 s; the first after `first_in` seconds (0: a whole period)
`kv_get(key, key_length, value, capacity)`	returns the value's length. `HOST_ERR_NOT_FOUND`; `HOST_ERR_NO_SPACE` for a buffer too small
`kv_set(key, key_length, value, length)`	a length of 0 removes the key. Refused for a `url` or `link` setting
`file_size(name, name_length)`	the file's length, or `HOST_ERR_NOT_FOUND`
`file_read(name, name_length, offset, bytes, capacity)`	returns how many bytes were read: fewer at the end, 0 past it
`file_write(name, name_length, offset, bytes, length)`	inside the file or right at its end. A length of 0 at offset 0 removes it

`poll_now(poll)`	fetch a poll of the manifest now
`poll_set_interval(poll, seconds)`	another pace for it, 10 s to a day, kept through a restart
`http_request(tag, method, url, url_length, headers, headers_length, body, body_length)`	`HOST_HTTP_GET`, `_POST`, `_PUT`, `_DELETE`, `_PATCH`, `_HEAD`, `_PROPFIND`, `_REPORT`; headers as "Name: value" lines
`data_value(index, text, capacity)`	inside `app_on_values()`: the value of path `index` as text, not terminated; returns its length
`data_response(response)`	inside a data handler, a `host_response_t`: status, length, retry_after, content_type[48], etag[64], last_modified[32], location[128], link[192]
`text_fill(pattern, pattern_length, arguments, count, text, capacity)`	the pattern with `{0}` .. `{3}` replaced by `host_text_argument_t` - number, or text and length; returns the length written
`void host_log(level, text, length)`	`HOST_LOG_ERROR`, `_WARN`, `_INFO`, `_DEBUG`; one line of up to 200 characters

21. Time

Call	What it does
`int64_t time_now(void)`	unix seconds; 0 while the time is not known
`time_zone_offset(void)`	seconds to add for the user's local time, daylight saving included
`time_local(parts), time_local_at(int64_t unix_time, parts)`	a `host_local_time_t`: year, month 1 - 12, day, weekday 0 = Sunday, hour, minute, second
`int64_t time_from_local(parts)`	the moment at which the user's clock shows this; a day past the end of its month counts on into the next
`time_text(int64_t unix_time, style, text, capacity)`	`HOST_TIME_CLOCK`, `_DATE`, `_WEEKDAY`, `_LONG_DATE`, `_NEAR`, `_NEAR_DAY`; in the device's language; returns the length
`int64_t time_uptime_ms(void)`	milliseconds since the device started; counts through sleep
`uint32_t time_boot_id(void)`	another number after every restart, never 0: keep it beside a stored uptime

22. The system API

What the host's own pages are written with. Any application may have it - the device is its owner's - by declaring `"system": true`; the page then tells the user before installing, and a module that imports a `sys_*` call without the declaration is refused.

Call	What it does
`sys_info(what, text, capacity)`	about the device, as text: `SYS_INFO_VERSION`, `_NAME`, `_NETWORK`, `_ADDRESS`, `_WIFI` ("online", "connecting", "setup"), `_SLOT`, `_FREE_MEMORY`, `_TIME_ZONE`, `_JOIN_CODE`
`sys_number_get(what, number), sys_number_set(what, value)`	a setting of the host that is a number, with its range: `SYS_NUMBER_TIME_AUTOMATIC`, `_REFRESH_FULL_EVERY`, `_REFRESH_MIN_SECONDS`, `_REFRESH_FORCE_HOURS`, `_POWER_SAVING`, `_REFRESH_FAST`, `_FONT`, `_MENU_IDLE_MINUTES`, `_HEADER_CLOCK`
`sys_text_set(what, text, length)`	one that is text: `SYS_TEXT_TIME_ZONE`, a POSIX TZ string
`sys_time_set(int64_t unix_seconds)`	the clock by hand; fails while it is set from the network
`sys_app_list(ids, capacity), sys_app_title(id, text, capacity), sys_app_remove(id)`	the installed applications by their ids, up to 16
`sys_app_default(void), sys_app_set_default(id)`	what the device shows by itself; 0 is none

`sys_logo(x, y, scale)`, `sys_logo_width(scale)`, `sys_logo_height(scale)`	the word perINKpheraL in its square letters, scale 1 - 5
`sys_do(action)`	`SYS_DO_RESTART`, `_FORGET_NETWORK`, `_FACTORY_RESET`. Ask the user first

23. The SDK library

Linked into every application by `mkapp`; what is not used is not kept.

Header and calls	What it is for
`app_text.h: app_text_start(t, buffer, size), app_text_add(t, text), app_text_add_number(t, n), app_text_add_padded(t, n, digits), app_text_length(text)`	building short strings without `printf`: always terminated, never past the buffer, cut between characters
`app_values.h: app_value_text(index, text, size), app_value_number(index, decimals, &n), app_parse_number(text, decimals, &n)`	inside `app_on_values()`: a picked value as terminated text, or as an integer with so many decimals kept
`json_pick.h: json_pick_init(p, paths, count), json_pick_feed(p, bytes, length), json_pick_finish(p), json_pick_value(p, index)`	the host's own picker, for answers that come in pieces: up to 16 paths, a value up to 63 bytes
`app_ui.h: app_list_button, app_list_first; app_picker_button, app_picker_draw; app_date_start, app_date_button, app_date_draw, app_date_seconds`	what the buttons do on the kit's list, picker and date; each button call answers `APP_UI_CHANGED`, `_DONE`, `_CANCELLED` or `_IGNORED`
`app_alloc.h: app_alloc(bytes), app_alloc_left(), app_alloc_mark(), app_alloc_release(mark)`	an arena over what the page has left: pieces one after another, given back to a mark. `NULL` when there is no room
`strings.h (made by mkapp): STR_*, app_string(id)`	the application's strings in the device's language
`pictures.h (made by mkapp): PICTURE_*`	the drawings of `pictures/`
`memcpy, memmove, memset, strlen`	what the compiler calls by itself

24. Limits

Name	Value	What it bounds
`HOST_API_VERSION`	2	the manifest's `abi`
`MANIFEST_MODULE_MAX_BYTES`	48 KB	`app.wasm`
`MANIFEST_NAME_MAX / _TITLE_MAX`	13 / 23 characters	an application's name and title
`MANIFEST_POLLS_MAX / _HOSTS_MAX / _SETTINGS_MAX`	4 / 4 / 8	what a manifest declares
`MANIFEST_URL_MAX / _SETTING_VALUE_MAX`	511 / 255 bytes	a poll's URL; a setting's value
`HOST_TEXT_MAX`	127 bytes	one text drawn or measured
`HOST_HEADER_TEXT_MAX / HOST_STATUS_TEXT_MAX`	63 / 31 bytes	a header text; a header status
`HOST_KV_KEY_MAX / _VALUE_MAX / _TOTAL_MAX`	15 / 1024 / 4096	a key; a value; an application's whole store
`HOST_FILE_NAME_MAX / HOST_FILES_MAX`	23 / 120 KB	a file's name; all files of an application
`HOST_DATA_CHUNK_MAX`	1024 bytes	a piece of an answer
`HOST_ANSWER_MAX / HOST_ANSWER_PIECES_MAX`	256 KB / 4 MB	an answer picked apart; an answer in pieces
`HOST_VALUE_MAX`	63 bytes	a picked value
`HOST_HTTP_HEADERS_MAX / _BODY_MAX / _PENDING_MAX`	1023 / 512 / 2	a request's headers and body; requests waiting on the device

`HOST_REQUEST_FIRST .. _LAST`	16 .. 255	tags of an application's own requests
`HOST_TIMER_COUNT / HOST_ALARM_COUNT`	4 / 4	per application
`HOST_ALARM_EVERY_MIN .. _MAX`	60 .. 86400 s	a repeating alarm
`HOST_TEXT_ARGUMENTS_MAX`	4	arguments of `text_fill()`
`HOST_QR_TEXT_MAX / HOST_LOG_TEXT_MAX`	200 / 200	a QR code's text; a log line
`HOST_ICON_CELLS / HOST_ICON_CELL_MAX`	11 / 16	an icon's grid; the largest cell in pixels
`strings of one language`	8 KB	the table the host loads

The header itself, `sdk/host_api.h`, is the contract and says more than a table can: when a call and this document disagree, the header is right and this document has a bug.